

Baza matematică

CAPITOLUL III

Cuprins

Introducere

Mulțimi

Logarimi

Sume și recurențe

Tehnici pentru demonstrații matematice

Inducție matematică și recursivitate

Exerciții

Introducere

Dezvoltarea și analiza algoritmilor presupune unele cunoștințe de **matematică**, legate de:

- Funcții, funcții logaritmice și limite
- Mulțimi și relații pe mulțimi
- Serii și sume
- Recursivitate
- Tehnici pentru demonstrații matematice

Mulțimi

Conceptul de **mulțime** are o aplicabilitate largă în domeniul calculatoarelor

O mulțime este o colecție de elemente **unice**

Elementele de obicei aparțin unui tip

Nu există conceptul de duplicare într-o mulțime

Orice valoare poate să aparțină sau să nu aparțină mulțimii

Mulțimi

Exemplu:

$M = \{ 1; 7; 23 \}$ – tipul întreg

$1 \in M$ – elementul 1 aparține mulțimii M

$2 \notin M$ – elementul 2 nu aparține mulțimii M

Mulțimi

Ex. $M = \{1; 7; 23\}$ $Q = \{1; 2; 23\}$ – tipul întreg

Reuniunea mulțimilor: $M \cup Q = \{1; 2; 7; 23\}$

Intersecția mulțimilor: $M \cap Q = \{1; 23\}$

Diferența mulțimilor: $M - Q = \{7\}$

$$Q - M = \{2\}$$

Mulțimea vidă $= \emptyset$

Familia tuturor submulțimilor: Pentru $M = \{1; 7; 23\}$, familia tuturor submulțimilor

$$P = \{\emptyset, \{1\}, \{7\}, \{23\}, \{1; 7\}, \{1; 23\}, \{7; 23\}, \{1; 7; 23\}\}$$

Mulțimi

O **secvență** = o colecție de elemente într-o anumită ordine, care se pot repeta.

Atenție: secvența **poate conține elemente duplicate**.

Secvența $\langle 1, 2, 3 \rangle$ este diferită de secvența $\langle 1, 3, 2 \rangle$

Mulțimi

O **relație** R definită pe o mulțime S , este un set de perechi ordonate, formate din elemente ale lui S

Exemplu

$$S = \{a, b, c\}$$

$$R1 = \{ \langle a, c \rangle, \langle b, c \rangle, \langle c, b \rangle \}$$
 este o relație

$$R2 = \{ \langle a, a \rangle, \langle a, c \rangle, \langle b, b \rangle, \langle b, c \rangle, \langle c, c \rangle \}$$
 este o altă relație

Notăția xRy , ne arată ca elementele $\langle x, y \rangle$ sunt în relația R

Ex: $2 \leq 3$, $\langle 2, 3 \rangle$ sunt în relația mai mic sau egal (sau 2 mai mic sau egal cu 3)

Mulțimi

O **relație** poate fi:

- Reflexivă – dacă aRa pentru oricare $a \in S$
- Simetrică – dacă aRb atunci și bRa , pentru oricare $a \in S$
- Antisimetrică – dacă aRb și bRa , atunci $a=b$, pentru oricare $a, b \in S$
- Tranzitivă – dacă aRb și bRc , atunci aRc , pentru oricare $a, b, c \in S$

O relație este una de **echivalență** dacă este **reflexivă, simetrică și tranzitivă**

Exemplu

- Pentru întregi = este o relație de echivalență
 1. $a=a$
 2. Dacă $a=b$ atunci $b=a$
 3. Dacă $a=b$ și $b=c$ atunci $a=c$

Logaritmi

Logaritm in baza b din y este puterea la care trebuie ridicat b ca să obținem valoarea y

$$\log_b y = x$$

Dacă $\log_b y = x$ atunci $b^x = y$ și $b^{\log_b y} = y$

Proprietăți:

$$\log_A B = \frac{\log_C B}{\log_C A}, \quad A, B, C > 0, A \neq 1$$

$$\log (AB) = \log A + \log B, \quad A, B > 0$$

$$\log (A/B) = \log A - \log B, \quad A, B > 0$$

$$\log (A^B) = B \log A, \quad A, B > 0$$

$$\log 1 = 0$$

Logaritmi

În domeniul calculatoarelor și tehnologiei informației se folosește cu preponderență **baza 2** pentru logaritmi

Ex: Care este numărul minim de biți necesari pentru a reprezenta n valori distincte

Răspuns $\lceil \log_2 n \rceil = \text{ceiling}(\log_2 n)$

Pentru 1000 de valori, avem nevoie de cel puțin 10 biți , $\lceil \log_2 1000 \rceil = 10$, $2^{10} = 1024$

Sume și recurențe

Sume și recurențe:

Exemplu de utilizare: Când analizăm timpul de rulare pentru un program care conține bucle, trebuie să adunăm timpii de rulare pentru fiecare iterație.

Notăție:

$$\sum_{i=1}^n f(i)$$

Suma valorilor funcției f , pe un interval de valori întregi ($i = \overline{1, n}$)

Sume și recurențe

De obicei se dorește înlocuirea sumei cu o ecuație algebrică echivalentă, proces numit **rezolvarea** sumei

Exemple:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$
$$\sum_{i=1}^n i^2 = \frac{2n^3 + 3n^2 + n}{6} = \frac{n(2n+1)(n+1)}{6}$$
$$\sum_{i=1}^{\log n} n = n \log n$$

Sume și recurențe

$$\sum_{i=0}^{\infty} a^i = \frac{1}{1-a} \text{ pentru } 0 < a < 1$$

$$\sum_{i=0}^n a^i = \frac{a^{n+1} - 1}{a - 1} \text{ pentru } a \neq 1$$

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

$$\sum_{i=0}^{\log n} 2^i = 2^{\log n + 1} - 1 = 2n - 1$$

Sume și recurențe

$$\sum_{i=1}^n \frac{1}{2^i} = 1 - \frac{1}{2^n}$$
$$\sum_{i=1}^{\log_2 n} \frac{1}{2^i} = 2 - \frac{n+2}{2^n}$$

Echivalențele pot fi demonstrate prin **inducție matematică**

Sume și recurențe

În matematică se spune că un **șir** a_n este definit printr-o **relație de recurență** dacă fiecare termen al acestuia poate fi scris ca o funcție de termeni anteriori

Exemplu funcția factorial:

$$\begin{cases} n! = (n-1)! \cdot n, \text{ pentru } n > 1 \\ 1! = 0! = 1 \end{cases}$$

Funcția Fibonacci

$$\begin{cases} \text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2), \text{ pentru } n > 2 \\ \text{Fib}(1) = \text{Fib}(0) = 1 \end{cases}$$

Sume și recurențe

Exemplu de utilizare: Pentru calculul timpului de rulare a unei funcții recursive

Ex. funcția factorial

Pentru cazurile de baza 0 și 1, durata funcției este constantă, în rest timpul poate fi modelat prin ecuația:

$T(n) = T(n - 1) + c$, pentru $n > 1$, $T(0) = T(1) = c$, unde $c = \text{constantă}$, și $T(n)$ costul apelului pentru valoarea n

Ca în cazul sumei, dorim să reducem ecuația la o formă compactă

Sume și recurențe

Expandăm ecuația

$$\begin{aligned}T(n) &= T(n-1) + c \\&= (T(n-2) + c) + c \\&= T(n-2) + 2c \\&= T(n-3) + 3c \\&\quad \dots \\&= T(1) + (n-1)c \\&= nc\end{aligned}$$

Formula se demonstrează prin **inducție matematică**

Tehnici pentru demonstrații matematice

Rezolvarea unei probleme are două părți: **investigarea** și **demonstrația**

Prin **investigare** căutăm o soluție și o dată gasită aceasta trebuie demonstrată ca fiind soluția corectă pentru toate cazurile vizate

Pentru **demonstrarea** unei soluții matematice avem o serie de metode standard

Cele mai folosite tehnici:

- Deducția, demonstrarea **directă**
- Demonstrarea prin **contradicție**
- **Inducția matematică**

Tehnici pentru demonstrații matematice

Demonstrarea directă

- Prin deducție logică, folosind logica matematică
- Ex: pentru a demonstra că două propoziții matematice P și Q sunt echivalente, se poate demonstra că P implică Q și Q implică P

Tehnici pentru demonstrații matematice

Demonstrarea prin **contradicție**, demonstrația indirectă

- Demonstrația prin contradicție este o formă de demonstrație care stabilește adevărul sau validitatea unei propoziții
- Demonstrarea prin contradicție, pleacă de la ipoteza că teorema este **falsă**, apoi folosind logica arată că asumarea propoziției ca fiind falsă duce la o **contradicție**
- Pentru a demonstra că o soluție nu este corectă ajunge să aducem un contraexemplu
- Ca urmare, niciun număr de exemple pozitive nu pot demonstra o teoremă

Tehnici pentru demonstrații matematice

Exemplu

- Propoziția: Nu există o valoare maximă pentru numerele naturale
- Demonstrație:

Pasul 1 – Negarea propoziției și asumarea ei ca ipoteză. Există o valoare maximă pentru numerele naturale (o notăm cu M)

Pasul 2 – Demonstrăm că ipoteza duce la o contradicție.

$N = M+1$, N este tot un număr natural pentru că este suma a două numere naturale

$N > M \Rightarrow$ Ipoteza este falsă

Tehnici pentru demonstrații matematice

Inducția matematică

Este o modalitate de demonstrație utilizată în matematică pentru a stabili dacă o anumită propoziție este valabilă pentru un număr nelimitat de cazuri, contorul cazurilor parcurgând toate **numerele naturale**

Poate fi folosită pentru o gamă largă de teoreme

Este folosită în recursivitate

Conține doi pași principali: **cazul inițial** și **pasul de inducție**

Tehnici pentru demonstrații matematice

Fie Trm teorema de demonstrat pentru un parametru pozitiv n . Prin inducție matematică se va demonstra că Trm este adevărată pentru orice valoare n , pentru $n > c$ (unde c este o constantă), dacă următoarele condiții sunt adevărate:

1. Cazul inițial: $Trm(c)$ – este adevărată
2. Pasul de inducție: Dacă $Trm(n-1)$ este adevărată, atunci și $Trm(n)$ este adevărată

Pentru pasul 2, avem varianta de inducție puternică (strong induction) prin care demonstrăm că dacă $Trm(k)$ este adevărată pentru oricare k , $c \leq k \leq n$, atunci $Trm(n)$ este adevărată.

Inducție matematică și recursivitate

Recursivitatea este o metodă de rezolvare a problemelor, în care găsirea soluției se bazează pe împărțirea problemei în instanțe mai simple

În oricare din variantele inducției, avem o asemănare puternică între demonstrarea prin inducție și **recursivitate**:

- Ambele au cazuri simple/ de bază
- Ambele se bazează pe instanțe simplificate ale aceleași probleme

Inducție matematică și recursivitate

Exemplu

Suma lui Gauss

$$S(n) = n(n+1)/2, \text{ pentru } n \geq 0$$

Pasul 1: Caz de baza, $n=1$, $S(1)=1(1+1)/2=1$ (Adevarat)

Pasul 2: $S(n-1) \rightarrow S(n)$

$$S(n-1) = (n-1)(n-1+1)/2 = (n-1)n/2$$

$$\text{Dar } S(n) = S(n-1) + n$$

$$S(n) = (n-1)n/2 + n = (n^2 - n + 2n)/2 = n(n+1)/2$$

Inducție matematică și recursivitate

Exemple

Demonstrația $T(n) = n \cdot c$, pentru timpul de execuție al funcției factorial recursive, pentru $n \geq 1$

$$n! = \begin{cases} 1, & n = 1 \\ n * (n - 1)!, & n > 1 \end{cases}$$

Pasul 1: $T(1) = c$, caz de bază

Pasul 2: $T(n-1) \rightarrow T(n)$, (daca $T(n-1) = (n-1)c \Rightarrow T(n) = nc$)

$T(1)=c, T(2)=c+c=2c, \dots T(k)=kc \Rightarrow$

$T(n-1) = (n-1)c$

$T(n) = T(n-1)+c$, (din definitie)

$T(n) = (n-1)c + c = n \cdot c$

```
long factorial(int n)
{
    if (n == 1)
        return 1; //conditia de oprire
    else
        return(n * factorial(n - 1));
}
```

Inducție matematică și recursivitate

În mod asemănător cu inducția matematică și recursivitatea conține două cazuri (doi pași):

- Cazul de bază
 - rezolvă problema pentru cel mai mic/ cel mai simplu set de date
- Cazul recursiv
 - definește ipoteza (pentru inducție matematică) - presupunem rezolvă problema prin apelul funcției pe un set de date restrâns (simplificat)
 - bazat pe ipoteză, combină apelurile pe seturile restrânse de date, pentru a rezolva problema pentru setul de date de intrare dat

Inducție matematică și recursivitate

Ex1: Să se scrie o funcție recursivă care afișează în ordine crescătoare numerele naturale din intervalul [start, end], unde inceput și sfârșit sunt parametri de intrare pentru funcția dată:

```
void printAsc(int start, int end);
```

```
printAsc(1,4) => 1 2 3 4
```

Inducție matematică și recursivitate

Determinarea dimensiunii setului de date

$\text{end} - \text{start} + 1$

Pasul 1: determinarea cazului de baza

$\text{start} == \text{end}$

Pasul 2:

- Definirea ipotezei: Dacă apelăm funcția pe un set interval restrâns de date, funcția va afișa numerele din acel interval în ordine crescătoare
- Bazat pe ipoteză, se implementează soluția pentru setul de date de intrare

Inducție matematică și recursivitate

```
//Varianta 1
void printAsc(int start, int end)
{
    if (start == end)
        printf("%d ", start); //cazul de baza
    else if (start < end)
    {
        printf("%d ", start);
        printAsc(start+1, end); //ipoteza/apelul recursiv
    }
}
```

Inducție matematică și recursivitate

```
//Varianta 2
void printAsc(int start, int end)
{
    if (start == end)
        printf("%d ", start); //cazul de baza
    else if (start < end)
    {
        printAsc(start, end - 1); //ipoteza/apelul recursiv
        printf("%d ", end);
    }
}
```

Inducție matematică și recursivitate

```
//Varianta 3
void printAsc(int start, int end)
{
    if (start == end)
        printf("%d ", start); //cazul de baza
    else if (start < end)
    {
        printAsc(start, (start+end)/2); //ipoteza/apelul recursiv
        printAsc((start + end) / 2 + 1, end); //ipoteza/apelul recursiv
    }
}
```

Exerciții

Ex1: Pentru fiecare din următoarele relații explicați de ce se respectă/nu se respectă proprietățile de reflexivitate, simetrie, asimetrie și tranzitivitate

- "EsteFrateleLui" pentru un set de persoane
- $R = \{ \langle x, y \rangle \mid x^2 = y^2 \}$ pentru numere reale
- $R = \{ \langle x, y \rangle \mid x \bmod y = 0 \}$ pentru $x \in \{1, 2, 3, 4\}$

Exerciții

Ex2: Pentru următoarele relații să se demonstreze fie că este o relație de echivalență, fie că nu este:

- Pentru numere întregi, $a \equiv b$, dacă și numai dacă $a+b$ este par
- Pentru numere întregi, $a \equiv b$, dacă și numai dacă $a+b$ este impar
- Pentru numere raționale, $a \equiv b$, dacă și numai dacă $a-b$ este un întreg

Exerciții

Ex3: Să se definească un TDA pentru o mulțime cu elemente întregi

Ex4: Rescrieți funcția factorial astfel încât să nu fie recursivă

Ex5: Demonstrați prin inducție formulele din secțiunea "Sume și recurențe"

Ex6: Să se demonstreze folosind tehnica inducției puternice că orice număr natural n (strict pozitiv) poate fi scris sub forma

$$n = 2^i \cdot j, \quad \text{unde } i \text{ este un număr întreg } \geq 0 \text{ și } j \text{ este un număr natural impar}$$

Bibliografie selectivă

- Weiss, M. A. (1995). *Data structures and algorithm analysis*. Benjamin-Cummings Publishing Co., Inc..
- Shaffer, C. A. (2012). *Data structures and algorithm analysis*.
- Introduction to Data Structures, NYUx, edx.org (2022)